

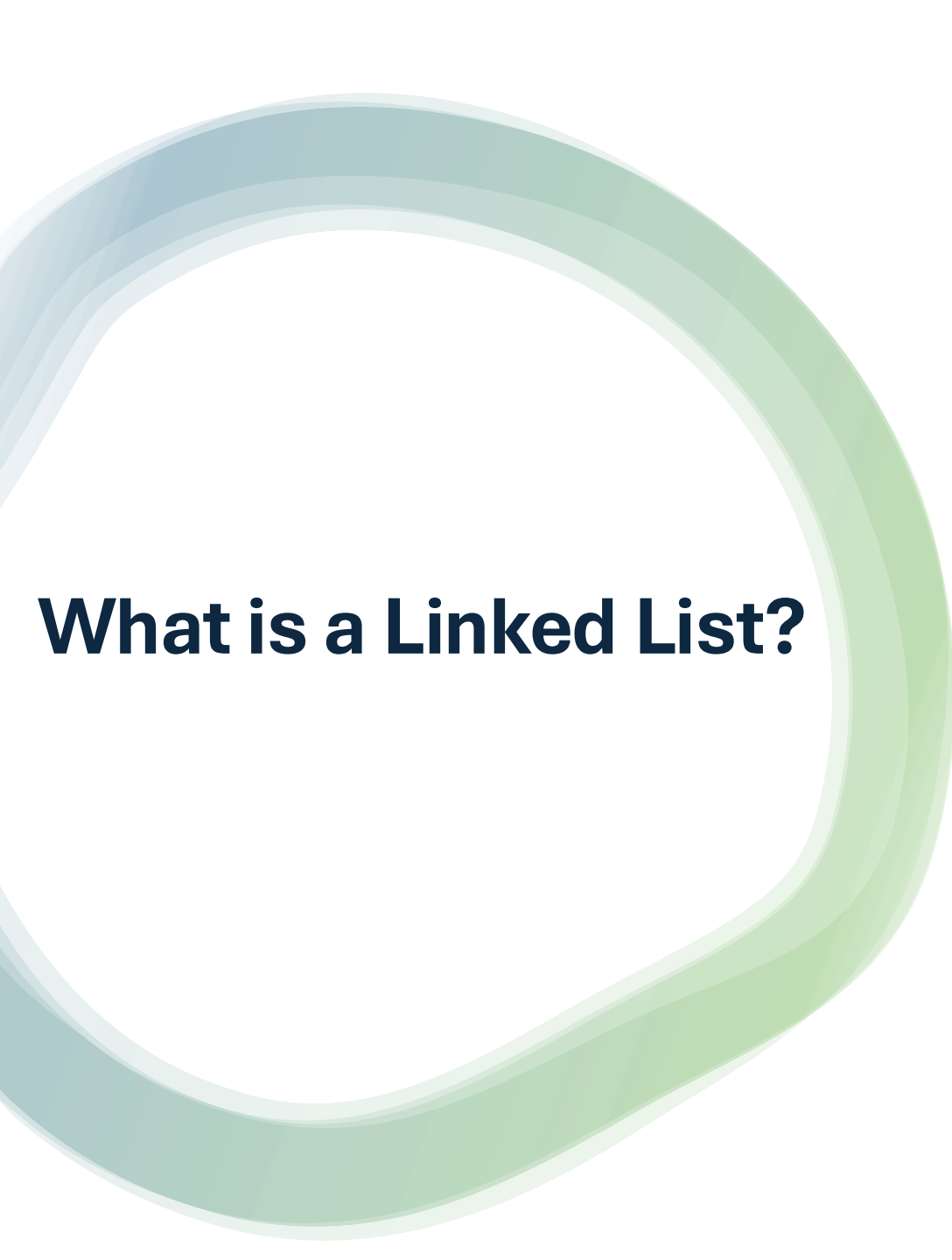
Doubly Linked List

Submitted by,

Name: Zannatul Ferdous Sinthia

Id: 2023-2-60-038

Course Code : CSE207



What is a Linked List?

A linked list is a type of linear data structure used to store a collection of elements, where each element (called a node).

Node contains two parts:

Data – the actual value or information stored.

Pointer – the address of the next node in the sequence.

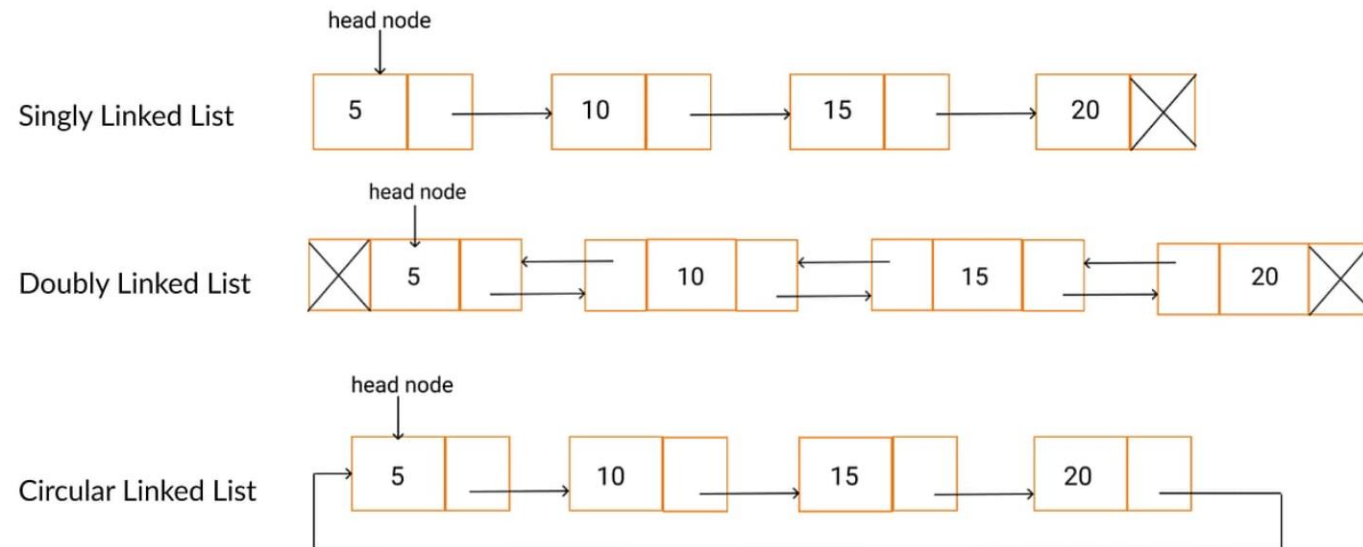
Unlike arrays, linked lists do not use contiguous memory locations. Instead, elements are connected using pointers, which makes insertion and deletion easier compared to arrays.

Types of Linked List

Singly Linked List – Each node points to the next node, last node points to NULL.

Doubly Linked List – Each node has two pointers: one to the next node and one to the previous node.

Circular Linked List – The last node points back to the first node, forming a circle.



What is a Doubly Linked List?

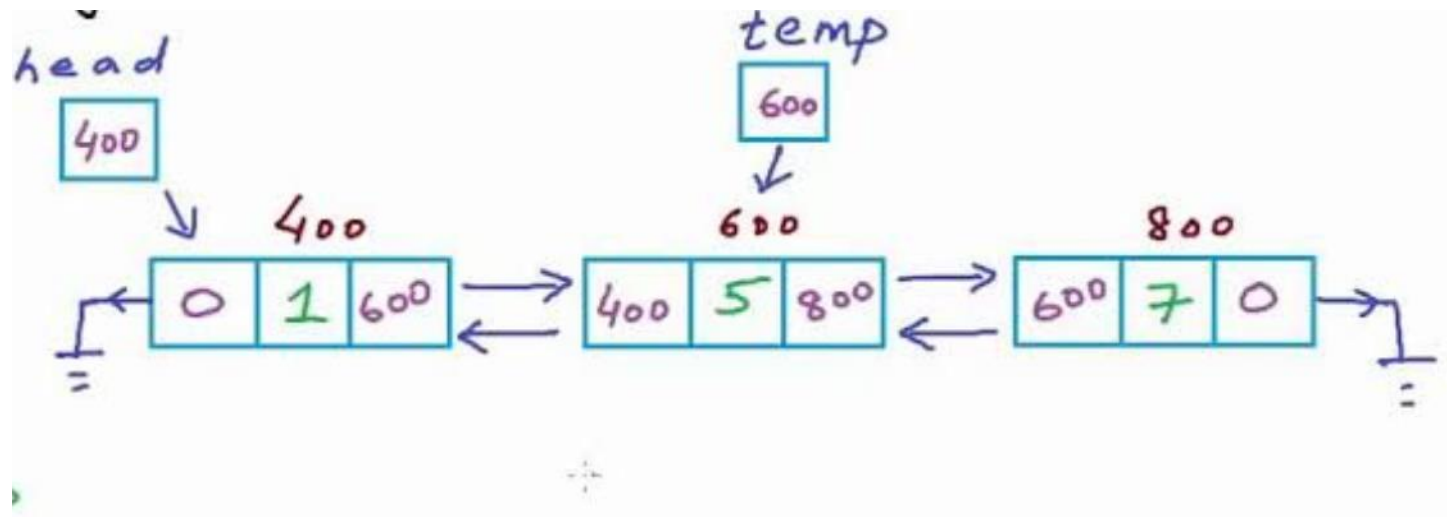
A **doubly linked list** is a linear data structure in which each node contains data and two pointers: one pointing to the next node and another pointing to the previous node, allowing traversal in both directions. 

Doubly Linked List in Data Structure



Structure of a Node in C

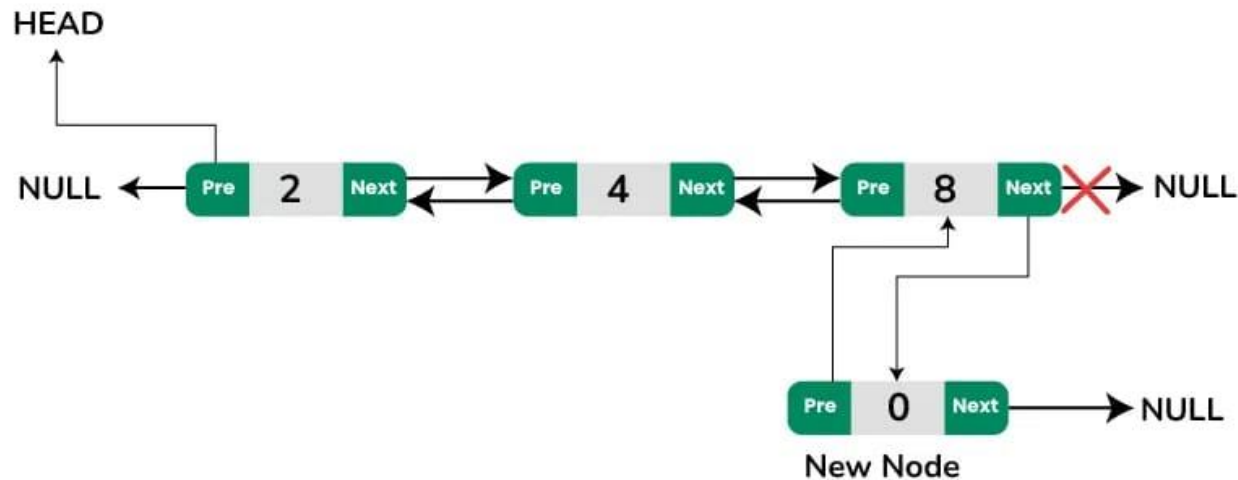
```
struct Node {  
    int data;  
    struct Node* prev;  
    struct Node* next;  
};
```



Initializing the list

```
Struct Node* head = Null;
```

Here, we declare a pointer named head of type **struct Node** and initialize it to **NULL**. This is the standard way to create an empty list in C.



A doubly linked list, it doesn't contain any nodes. To represent this empty state, we use a special pointer, often called the head pointer. This pointer holds the memory address of the very first node in the list. Since there are no nodes yet, the head pointer is set to NULL. This NULL value serves as a sentinel, indicating that the list is at its starting point and there's nothing to traverse.

Insertion Types

1

At Beginning:
Adjust head
pointer and prev of
old head.

2

At End: Traverse to
last node, adjust
pointers.

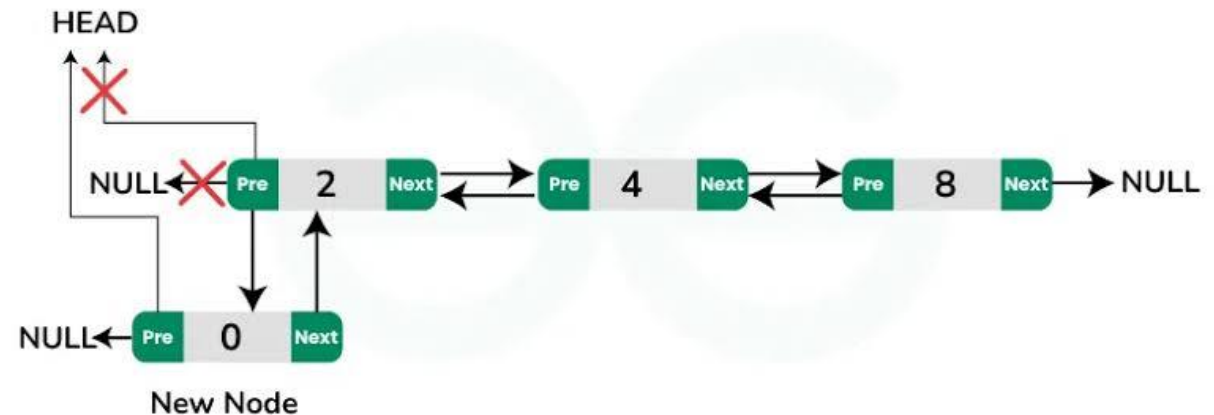
3

At Specific
Position: Link new
node
between prev and
next nodes.

Insertion at the Beginning (Step-by-Step)

Steps:

- Create a new node.
- Make the new node's next pointer point to the current head.
- Update the old head's prev pointer to point to the new node.
- Make the new node the head.



Insertion at the Beginning in Doubly Linked List

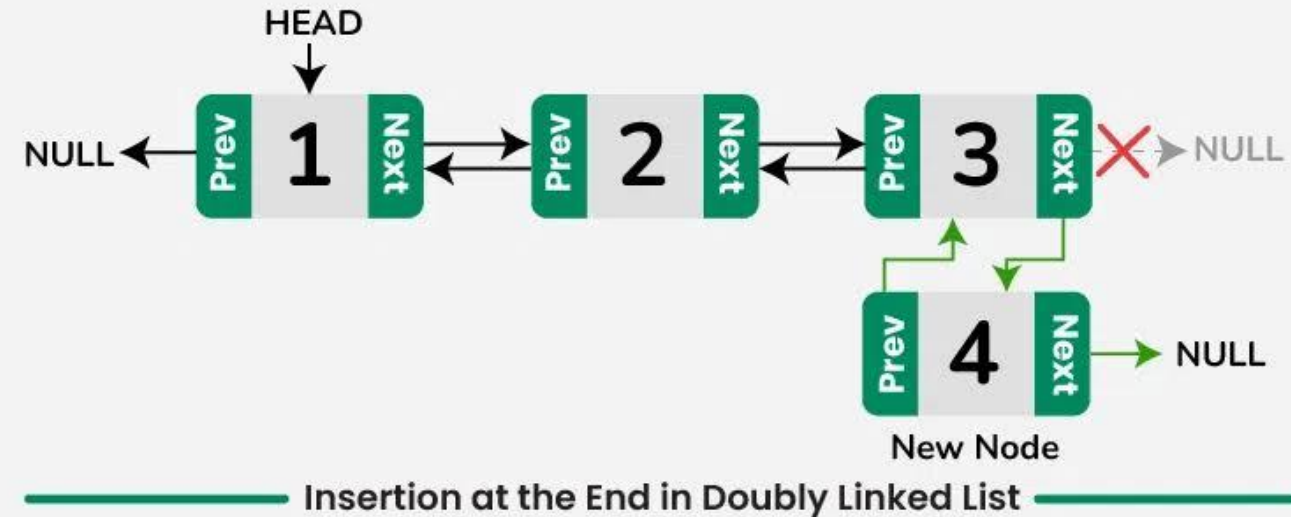
Code for Insertion at Beginning

```
void insertAtBeginning(struct Node** head_ref, int new_data) {  
  
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));  
    new_node->data = new_data;  
    new_node->next = (*head_ref);  
    new_node->prev = NULL;  
    if ((*head_ref) != NULL) {  
        (*head_ref)->prev = new_node;  
    }  
    (*head_ref) = new_node;  
}
```

Insertion at the End

Steps:

- Create a new node.
- Traverse to the last node.
- Update the last node's next pointer to the new node.
- Set the new node's prev pointer to the last node.



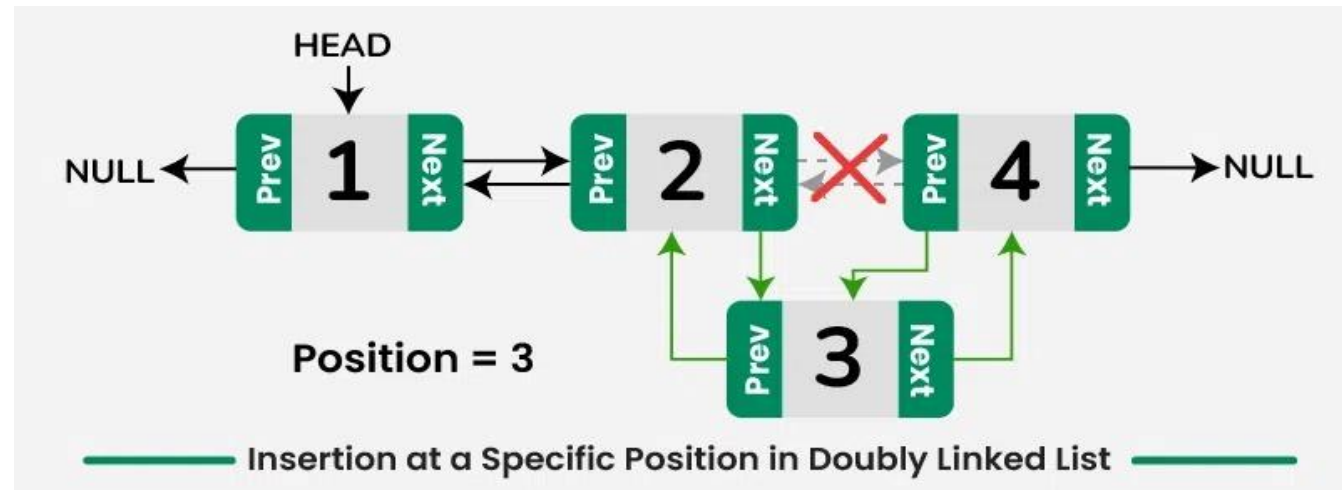
Code for Insertion at End

```
void insertAtEnd(struct Node** head_ref, int new_data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    struct Node* last = *head_ref;
    new_node->data = new_data;
    new_node->next = NULL;
    if (*head_ref == NULL) {
        new_node->prev = NULL;
        *head_ref = new_node;
        return;
    }
    while (last->next != NULL) {
        last = last->next;
    }
    last->next = new_node;
    new_node->prev = last;
}
```

Insertion at a Specific Position

Steps:

- Create a new node.
- Find the previous node.
- Update pointers to link the new node between the previous and next nodes.



Code for Insertion at Specific point

```
void insertAfter(struct Node* prev_node, int new_data) {
    if (prev_node == NULL) {
        printf("The given previous node cannot be NULL.");
        return;
    }

    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));

    new_node->data = new_data;

    new_node->next = prev_node->next;
    new_node->prev = prev_node;

    prev_node->next = new_node;

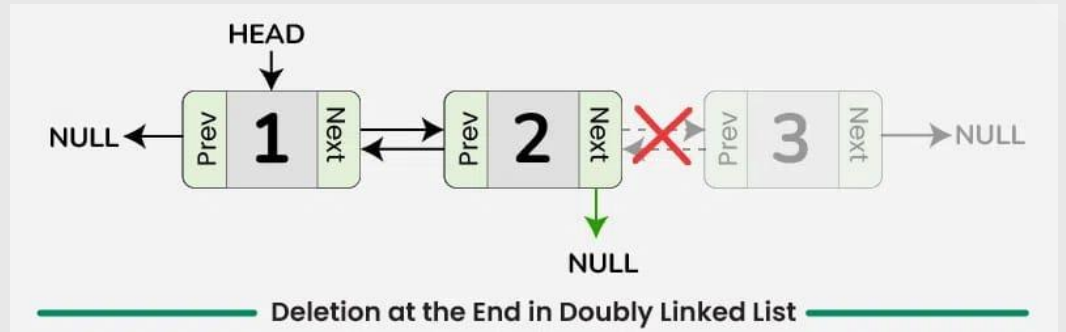
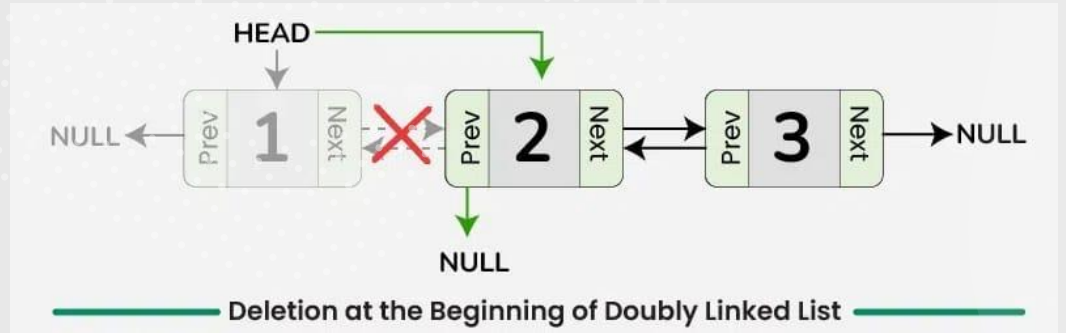
    if (new_node->next != NULL) {
        new_node->next->prev = new_node;
    }
}
```

Deletion Types

Delete from Beginning: Update head to next node, free old head.

Delete from End: Traverse to last node, free it.

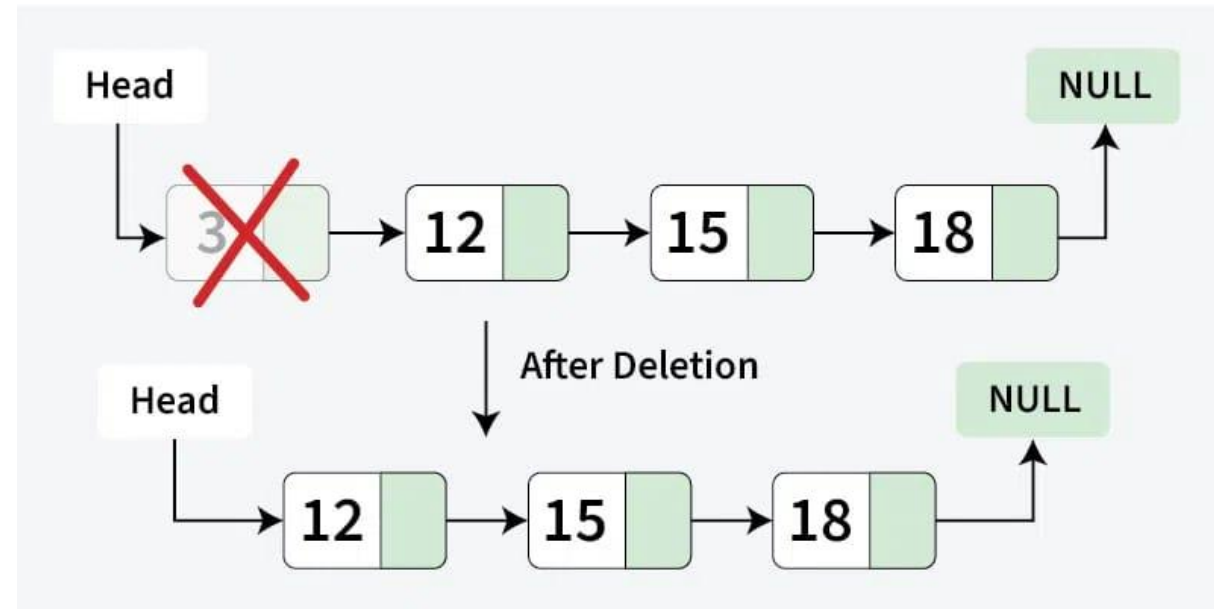
Delete Specific Node: Adjust prev and next pointers, free node.



Deletion of a Node (Step-by-Step)

Steps:

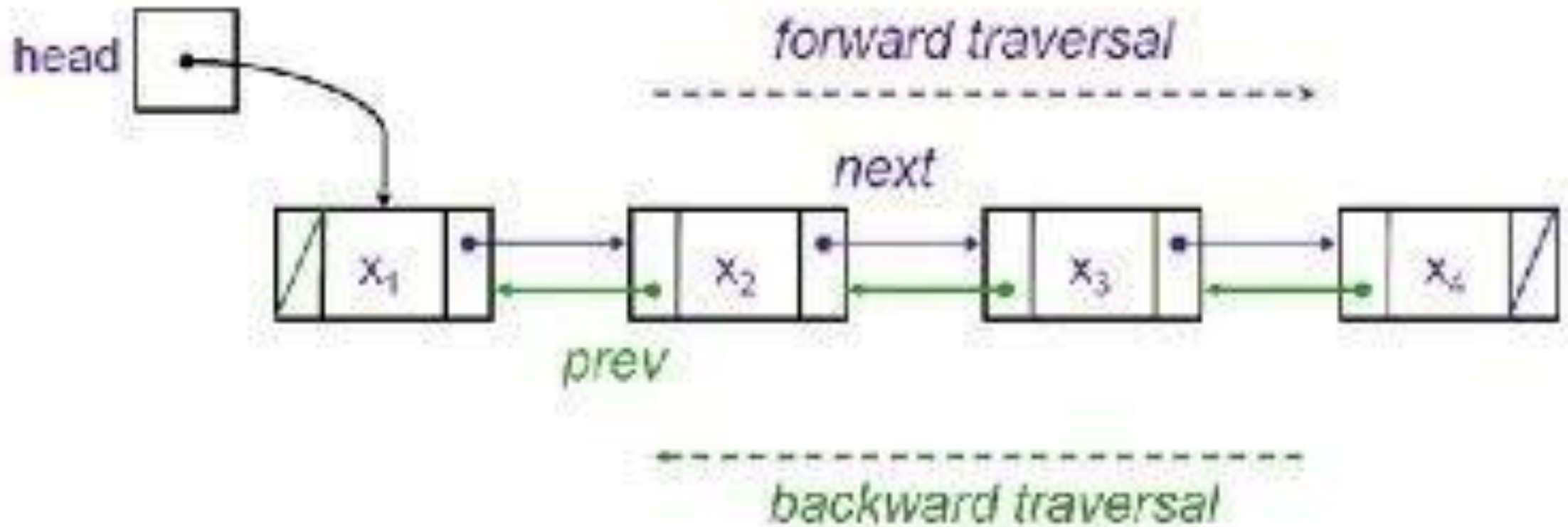
- Find the node to be deleted.
- Adjust the next pointer of the previous node.
- Adjust the prev pointer of the next node.
- Free the memory of the deleted node.



Code for Deletion of a Node

```
void deleteNode(struct Node** head_ref, struct Node* del_node) {  
    if (*head_ref == NULL || del_node == NULL) return;  
    if (*head_ref == del_node) {  
        *head_ref = del_node->next;  
    }  
    if (del_node->next != NULL) {  
        del_node->next->prev = del_node->prev;  
    }  
    if (del_node->prev != NULL) {  
        del_node->prev->next = del_node->next;  
    }  
    free(del_node);  
}
```


Traversal



Forward Traversal

```
void forwardTraversal(struct Node* node) {  
    while (node != NULL) {  
        printf("%d ", node->data);  
        node = node->next;  
    }  
}
```

Backward Traversal

```
void backwardTraversal(struct Node* last_node) {  
    while (last_node != NULL) {  
        printf("%d ", last_node->data);  
        last_node = last_node->prev;  
    }  
}
```

Advantages & Disadvantages

Advantages:

- Bidirectional traversal is possible.
- Deletion of a given node is more efficient.
- Can implement more advanced data structures.

Disadvantages:

- Increased memory usage per node.
- Slightly more complex implementation for managing pointers.



THANK YOU

Instructor :MD. MANOWARUL ISLAM

Course Code: CSE207

Section : 10